

Otimização

4.3.3 Restrições na prática: penalizar, rejeitar ou comparar

José Firmino Aguiar Madeira

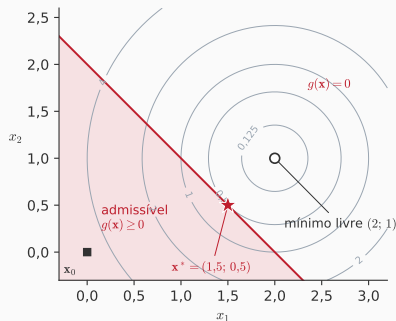
Um problema, cinco tratamentos

$$\min f(\mathbf{x}) = (x_1 - 2)^2 + (x_2 - 1)^2 \quad \text{s.a.} \quad g(\mathbf{x}) = 2 - x_1 - x_2 \geq 0$$

- $\mathbf{x}^* = (1,5; 0,5)$, $f^* = 0,5$, $u^* = 1$ (KKT, 4.2); o mínimo livre $(2; 1)$ é **inadmissível**.
- Violação: $v(\mathbf{x}) = -\langle g(\mathbf{x}) \rangle = \max(0, -g(\mathbf{x}))$, como em 4.3.1.

O **mesmo** Hooke–Jeeves (3.2.5), de $\mathbf{x}^{(0)} = (0; 0)$, com a restrição comunicada de cinco formas:

- A** ignorar • **B** penalização quadrática • **C** penalização exata •
D barreira extrema • **E** regra de admissibilidade



O problema é o mesmo; só muda a forma como **comunicamos as restrições ao algoritmo**.

O algoritmo sabe que há uma restrição?

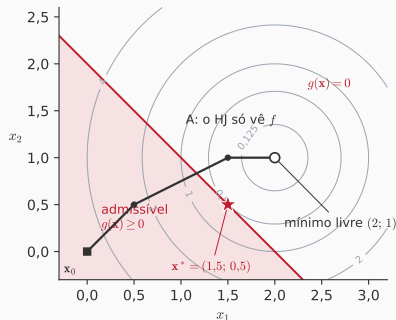
A — ignorar. O HJ recebe só $F = f$ e devolve

$\mathbf{x} = (2; 1)$, $g = -1$: inadmissível ($n_F = 96$).

- O HJ (e todo o cap. 3) minimiza o **número que lhe damos**; não sabe o que é g .
- A restrição só existe para ele se estiver **nesse número** — ou **nas comparações** que faz.
- O algoritmo fica **sem restrições**; a admissibilidade é uma **camada exterior**.

n_F : chamadas à função (ou ao par) que o HJ recebe;

$a = 2$, $P_0 = 0,5$, $T = 10^{-6}$ em todo o deck.



Podemos cobrar pela violação?

B — penalização quadrática, com R fixo:

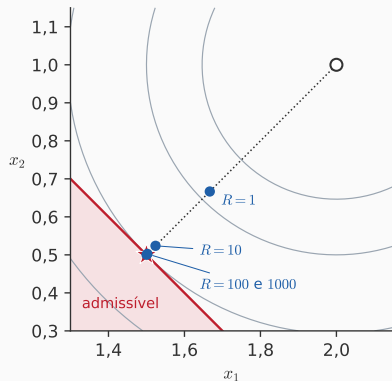
$$F_R(\mathbf{x}) = f(\mathbf{x}) + R \langle g(\mathbf{x}) \rangle^2 = f(\mathbf{x}) + R v(\mathbf{x})^2.$$

Do lado inadmissível, $\nabla F_R = \mathbf{0}$ dá

$$\mathbf{x}_R = (2; 1) - R v_R (1; 1), \quad v_R = \frac{1}{1 + 2R}.$$

- $v_R > 0$ para **todo** o R finito: sempre (ligeiramente) **inadmissível**.
- $\mathbf{x}_R \rightarrow \mathbf{x}^*$ **por fora**: é a penalização de 4.3.1, mas com R fixo e uma só minimização.
- O HJ encontra estes $\mathbf{x}_R$.

Pagar pela violação **não a proíbe**.



Quanto deve valer R ?

R	$\mathbf{x}_R$ (HJ)	$v(\mathbf{x}_R) = 1/(1 + 2R)$	$\ \mathbf{x}_R - \mathbf{x}^*\ $	n_F
1	(1,667; 0,667)	0,333	0,236	253
10	(1,524; 0,524)	0,048	0,034	244
100	(1,502; 0,502)	0,0050	0,0035	311
1000	(1,500; 0,500)	0,0005	0,00035	644

R pequeno

Barato, mas **visivelmente inadmissível**: com $R = 1$, $v = 1/3$.

R grande

Quase admissível, mas F_R tem um **vale estreito** ($\kappa \sim R$, 4.3.1): o HJ “rasteja” — 644 chamadas.

O R “certo” depende da **escala** de g (4.3.1). A cura de 4.3.1 é R **crescente**, com arranque a quente; com R fixo, escolhe-se entre inadmissível e mal condicionado.

Podemos tornar a penalização exata?

C — penalização exata (ℓ_1): paga-se v , não v^2 :

$$F_R = f + R v = f + R |\langle g \rangle|.$$

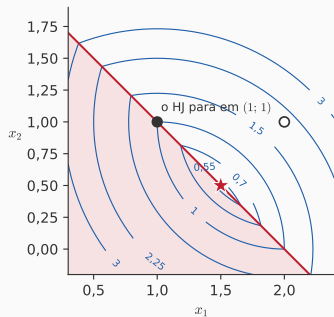
Um resultado local

Se $\mathbf{x}^*$ é minimizante local **estrito** e satisfaz as KKT com multiplicadores u_j^* , então $\mathbf{x}^*$ é minimizante **local** de $f + R \sum_j |\langle g_j \rangle|$ para todo o $R > \max_j u_j^*$.

(Nocedal e Wright, 2006, Teor. 17.3)

Aqui $u^* = 1$. O HJ dá:

- $R = 0,5 < u^*$: (1,75; 0,75), inadmissível;
- $R = 1 = u^*$ (fora do teorema): aqui ainda dá $\mathbf{x}^* = (1,5; 0,5)$;
- $R = 2 > u^*$: o minimizante de F_R **é** $\mathbf{x}^*$... mas o HJ **para em (1; 1)**.



Níveis de F_2 : um **bico** em $g = 0$.

O preço: um majorante de u^* (desconhecido), e F_R **não diferenciável** onde está $\mathbf{x}^*$. Resultado **local**, sob hipóteses — não universal.

Porque parou o Hooke–Jeeves em (1; 1)?

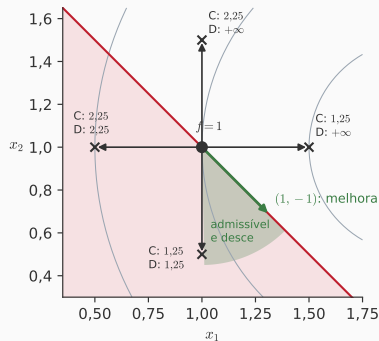
Em (1; 1), $f = 1$. As tentativas $+Pe_1$, $+Pe_2$ saem de X (C paga R v, D dá $+\infty$); $-Pe_1$, $-Pe_2$ ficam em X , mas f **aumenta**.

Para **qualquer** $P > 0$, nenhum $\pm e_j$ melhora: em D, as direções que descem e ficam admissíveis são a cunha verde; em C, as que descem F_2 são o quadrante $d_1 > 0$, $d_2 < 0$. O HJ divide P até T e para num ponto **não** ótimo.

Correção: mudar as direções, não o método

O mesmo HJ em y , com $x = Qy$, $Q = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$: direções **ao longo** e **na normal** à fronteira. C ($R = 2$), D e E chegam a x^* ($n_F = 208$ nos três).

Com restrições lineares, as direções devem acompanhar as restrições quase ativas (Lewis e Torczon, 2000).



E se simplesmente recusarmos pontos inadmissíveis?

D — barreira extrema:

$$F(\mathbf{x}) = \begin{cases} f(\mathbf{x}), & g(\mathbf{x}) \geq 0, \\ +\infty, & g(\mathbf{x}) < 0 \text{ (} f \text{ não é avaliada).} \end{cases}$$

- Sem R , sem escalas: um inadmissível **perde sempre**; f **nunca** é avaliada fora de X .
- Iteradas **admissíveis**, desde que $\mathbf{x}^{(0)} \in X$: de $(3; 3)$ tudo vale $+\infty$ e o HJ não sai do sítio.
- De $(0; 0)$: para em $(1; 1)$; $n_F = 91$, das quais 45 fora de X (sem avaliar f).
- Não é a barreira log (4.3.2): não há parede, há “não”.

```
function y = ExtremeBarrier(f, g, x)
if any(g(x) < 0)
    y = Inf;           % f nao e avaliada
else
    y = f(x);
end
end
```

É o que fazem o GLODS e o DMS

GLODS (5.1), MultiGLODS e DMS (6.5):
 $f = +\infty$ fora de X : a função objetivo **não é avaliada** nos pontos inadmissíveis.

E se nem sequer houver um $g(x)$?

Restrição explícita

Conhecemos g : sabemos **se** e **quanto** se viola.

Há v : B, C e E são possíveis.

Uma classificação das restrições em otimização por simulação: Le Digabel e Wild (2023).

```
function y = simulador(x)
if x(1) + x(2) > 2, y = NaN;
else, y = (x(1) - 2)^2 + (x(2) - 1)^2; end
end
F = @(x) ExtremeBarrier(@simulador, [], x);
x = HookeJeeves(F, [0 0], 2, 0.5, 1e-6);
```

Restrição oculta

O simulador **falha** (NaN) sem dizer porquê nem quanto. Não há v : das estratégias aqui consideradas, **só D** dispensa uma medida de violação.

```
% caixa negra
% "nao converge"
```

```
% NaN -> Inf
```

Resultado: **a mesma corrida** que D — $(1; 1)$, $n_F = 91$ — sem nunca conhecer g . As 45 falhas também **custam**: o simulador correu até falhar.

É preciso somar objetivo e violação?

E — regra de admissibilidade. O HJ só precisa de saber **qual de dois pontos é melhor**. Compara-se o par (v, f) — sem nenhuma função nova:

1. admissível **vence** inadmissível;
2. entre admissíveis, vence o de menor f ;
3. entre inadmissíveis, vence o de menor v .

É a regra de Deb (2000), a do NSGA-II (6.6).

- De $(0; 0)$: os mesmos pontos base e o mesmo $(1; 1)$ que D ($n_F = 91$).
- De $(3; 3)$: D não sai do sítio; E reduz v e chega a $(1; 1)$ ($n_F = 96$).
- Precisa de v : só para restrições explícitas (f só faz falta quando os v empatam).

MATLAB

```
FE = @(x) [v(x), f(x)];           % o par (v, f)
menor = @(a, b) a(1) < b(1) || ...
        (a(1) == b(1) && a(2) < b(2));
x = HookeJeeves(FE, [3 3], 2, 0.5, 1e-6, ...
               [], false, menor);
```

Python

```
FE = lambda x: (v(x), f(x))       # o par (v, f)
menor = lambda a, b: a[0] < b[0] or \
        (a[0] == b[0] and a[1] < b[1])
r = hooke_jeeves(FE, [3, 3], 2, 0.5, 1e-6,
                 menor=menor)
```

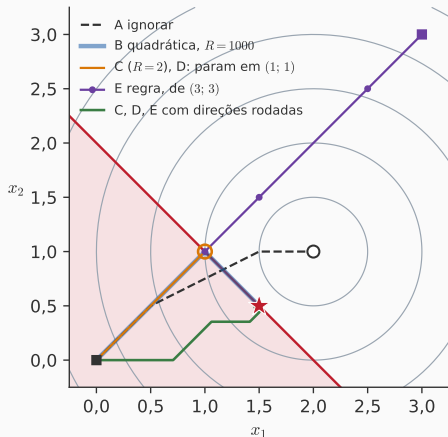
No HJ da UC, menor é opcional (por omissão, $a < b$).

O mesmo problema, cinco tratamentos

Tratamento	O que o HJ recebe	Resultado (n_F)	Admissível?	Lição
A ignorar	f	(2; 1) (96)	não	o algoritmo não vê g
B quadrática	$f + R v^2$, $R = 1000$	(1,500; 0,500), $v = 5 \cdot 10^{-4}$ (644)	não (quase)	aproxima por fora; R grande custa
C exata	$f + R v$, $R = 2$	(1; 1) (91)	sim	$\mathbf{x}^*$ minimiza F_R , mas há um bico
D barreira extrema	f , ou $+\infty$ sem avaliar f	(1; 1) (91)	sim	nunca sai de X ; precisa de $\mathbf{x}^{(0)} \in X$
E regra	comparações de (v, f)	(1; 1) (91); de (3; 3): (1; 1) (96)	sim	sem somar; arranca de fora
C, D, E rodados	as mesmas; direções $\mathbf{Qe}_j$	$\mathbf{x}^* = (1,5; 0,5)$ (208)	sim	as direções importam

B e C **mudam a função**; D **recusa** pontos; E **muda a comparação**. O algoritmo é sempre o mesmo.

As trajetórias



- Em X , todos veem a mesma f : de $(0; 0)$, os caminhos coincidem até à fronteira.
- Aí separam-se: A atravessa-a; B desliza por fora; C e D param em $(1; 1)$.
- E, de $(3; 3)$, primeiro reduz a violação; já em X , compara como D.
- Com as direções rodadas, segue-se a fronteira até x^* .

Para escolher

g explícita e barata $\rightarrow$ C ou E (ou B com R crescente, 4.3.1).

f cara, ou indefinida fora de $X \rightarrow$ D. Restrição oculta (sem v) $\rightarrow$ D.

No computador: os cinco tratamentos em MATLAB e Python

MATLAB

```
f = @(x) (x(1) - 2)^2 + (x(2) - 1)^2;
g = @(x) 2 - x(1) - x(2);           % g(x) >= 0
v = @(x) max(0, -g(x));             % violacao
FA = f;                             % A: ignorar
FB = @(x) f(x) + R*v(x)^2;          % B: quadratica
FC = @(x) f(x) + R*v(x);            % C: exata
FD = @(x) ExtremeBarrier(f, g, x);  % D: barreira
FE = @(x) [v(x), f(x)];             % E: o par

menor = @(a, b) a(1) < b(1) || ...
    (a(1) == b(1) && a(2) < b(2));

x = HookeJeeves(FD, [0 0], 2, 0.5, 1e-6);
x = HookeJeeves(FE, [0 0], 2, 0.5, 1e-6, ...
    [], false, menor);
```

Python

```
f = lambda x: (x[0] - 2)**2 + (x[1] - 1)**2
g = lambda x: 2 - x[0] - x[1]        # g(x) >= 0
v = lambda x: max(0.0, -g(x))        # violacao
FA = f                               # A: ignorar
FB = lambda x: f(x) + R * v(x)**2    # B: quadratica
FC = lambda x: f(x) + R * v(x)       # C: exata
FD = lambda x: extreme_barrier(f, g, x) # D
FE = lambda x: (v(x), f(x))          # E: o par

menor = lambda a, b: a[0] < b[0] or \
    (a[0] == b[0] and a[1] < b[1])

r = hooke_jeeves(FD, [0, 0], 2, 0.5, 1e-6)
r = hooke_jeeves(FE, [0, 0], 2, 0.5, 1e-6,
    menor=menor)
```

O script completo (tabela, v_R , restrição oculta, arranque em (3; 3), bloqueio, direções rodadas) está no código complementar da UC, com Colab.

O QUE DEVE FICAR — 4.3.3

1. Um método sem restrições só vê **números** (ou comparações): as restrições entram por uma camada exterior.
2. **Penalizar** (B, C) muda a função: a quadrática aproxima por fora e mal condiciona; a exata torna $\mathbf{x}^*$ minimizante local com $R > u^*$ finito (sob hipóteses), mas o algoritmo pode não o encontrar.
3. **Recusar** (D) mantém as iteradas admissíveis, desde que se parta de um ponto admissível; das estratégias aqui consideradas, é a única que dispensa uma medida de violação.
4. **Comparar** (E) dispensa R e somas, e arranca de pontos inadmissíveis.
5. Com funções não suaves ou barreiras, as **direções** do método decidem se se chega a $\mathbf{x}^*$.

Ponte: GLODS (5.1) e DMS (6.5) usam a barreira extrema; os AG (5.3) penalizam ou rejeitam; o NSGA-II (6.6) usa a regra de admissibilidade.

Para além da UC: a barreira progressiva

Entre D e E: aceita pontos inadmissíveis com violação abaixo de um limiar que **diminui** ao longo das iterações, e recusa os outros, como D (Audet e Dennis, 2009; MADS/NOMAD).

Disposição das turbinas de um parque eólico (IDMEC, projeto M4WIND)

Decisão as coordenadas (x_i, y_i) de cada uma das N turbinas.

Objetivo a energia anual produzida, descontando as perdas por esteira.

Restrições distância mínima entre turbinas — $g_{pq} = \|(x_p, y_p) - (x_q, y_q)\| - d_{\min} \geq 0$, uma por par: $J = N(N - 1)/2$ desigualdades ($N = 30$: 435); fronteira do terreno e zonas interditas (restrições geométricas sobre cada turbina).

O que muda face ao académico f é multimodal e não tem derivadas fiáveis; KKT à mão (2^J casos) é impensável. As restrições entram tipicamente por **penalização** (4.3.1), por barreira extrema/reparação ou pela regra de admissibilidade (4.3.3, 6.6), dentro de um método do cap. 5.

A informação que temos: valores de f (caros) e restrições *geométricas* baratas de verificar. Essa assimetria — f cara, g barata — decide o tratamento das restrições num problema real (com D, f nem é avaliada num inadmissível).

Quadro de consulta — capítulo 4

Que informação temos sobre f e sobre as restrições? Quanto custa obtê-la? O que garante?

Método	Informação	Como se resolve / custo	Garantia	Principal limitação
Lagrange (4.1)	$\nabla f, \nabla h_\ell$ analíticos	à mão: resolver $n + K$ equações	necessária sob independência dos ∇h_ℓ	só candidatos;
KKT (4.2)	$\nabla f, \nabla g_j, \nabla h_\ell$	à mão: até 2^J casos (KKT não é um algoritmo)	necessária sob LICQ; suficiente se convexo	sistema não linear enumeração exponencial à mão
Penalização exterior (4.3.1)	f, g_j, h_ℓ (+ gradientes, se o interno os usar)	$\sum_t n_f$ das minimizações internas	$\mathbf{x}_R \rightarrow \mathbf{x}^*$ quando $R \rightarrow \infty$ (sob hipóteses [†])	$\kappa \sim R$; solução ligeiramente inadmissível
Barreira log (4.3.2)	f, g_j ; ponto interior	idem, + Fase I	$\mathbf{x}_R \rightarrow \mathbf{x}^*$ quando $R \rightarrow 0$ (sob hipóteses [†]); $u_j = R/g_j$	só desigualdades; $\kappa \sim 1/R$
Na prática (4.3.3)	f, g_j ou só “falhou”	uma minimização	C: exatidão local se $R > u^*$ (sob hipóteses); D, E: depende do método	as direções

[†] f, g_j (h_ℓ) contínuas; $\mathbf{x}_R$ minimizantes globais de P , com $(\mathbf{x}_R)$ limitada; $\mathbf{x}^*$ único (sem unicidade: todo o ponto limite é mínimo global); na barreira, também $X = \text{fecho do interior}$. Para estudo. Na aula: as duas páginas seguintes.

O QUE DEVE FICAR — capítulo 4

1. As restrições mudam as **direções** em que podemos melhorar f .
2. Os **multiplicadores** dizem que restrições “contam” e quanto valem.
3. **CHARACTERIZAR** (4.1–4.2): Lagrange ($h = 0$: $\nabla f = \sum \lambda_\ell \nabla h_\ell$) e KKT ($g \geq 0$: $u \geq 0$, $ug = 0$) usam derivadas $\rightarrow$ condições de 1.^a ordem $\rightarrow$ candidatos; globalidade no caso convexo.
4. **CALCULAR** (4.3): exterior ($R \uparrow \infty$, $\mathbf{x}_R \rightarrow \mathbf{x}^*$ por fora) e interior ($R \downarrow 0$, por dentro) usam o cap. 3 repetidamente $\rightarrow$ custo = soma das minimizações internas $\rightarrow$ convergência à solução sob hipóteses.
5. Transformar o problema com restrições em problemas sem restrições tem um **preço**: o condicionamento.
6. **NA PRÁTICA** (4.3.3): penalizar, recusar ou comparar — o algoritmo é o mesmo.

A seguir (cap. 5): estes métodos dão mínimos locais. E se houver várias bacias?

Referências e recursos

Referências Lista completa: *Bibliografia da UC*, no fim do PDF da UC.

- Nocedal, J., Wright, S. J., *Numerical Optimization*, 2.^a ed., Springer, 2006 — sec. 17.2 (penalização exata, Teor. 17.3).
- Deb, K., *An efficient constraint handling method for genetic algorithms*, Computer Methods in Applied Mechanics and Engineering 186(2–4):311–338, 2000 — regra de admissibilidade.
- Lewis, R. M., Torczon, V., *Pattern search methods for linearly constrained minimization*, SIAM J. Optim. 10(3):917–941, 2000.
- Audet, C., Dennis, J. E., Jr., *A progressive barrier for derivative-free nonlinear programming*, SIAM J. Optim. 20(1):445–472, 2009.
- Le Digabel, S., Wild, S. M., *A taxonomy of constraints in black-box simulation-based optimization*, Optimization and Engineering, 2023 (doi:10.1007/s11081-023-09839-3).

Recursos

- Código complementar: `ex04_3_3_constraint_handling`, `ExtremeBarrier`; `HookeJeeves` (3.2.5) com menor opcional.
- MATLAB: `patternsearch` e `ga` aceitam restrições. NOMAD: barreira progressiva.